Manual de Git + Prisma Migrations

Flujo de trabajo en equipo: ramas, migraciones y sincronización.

1. Conceptos clave

Herramienta	Qué hace
`git branch`	Crea ramas de trabajo aisladas para cada feature o fix
`git push/pull`	Sube y baja cambios del repositorio remoto (GitHub/GitLab)
`prisma migrate dev`	Crea una nueva migración a partir de cambios en `schema.prisma`
`prisma migrate deploy`	Aplica migraciones existentes a la base de datos
`prisma generate`	Regenera el cliente de Prisma en `generated/prisma/`
`prisma db push`	Sincroniza el schema sin crear migración (solo prototipado rápido)

2. Quién crea la migración (el que cambia el schema)

Cuando **tú** modificas el `schema.prisma` (agregas un campo, tabla o relación):

Paso	Comando	Descripción
1	`git checkout -b feature/mi-feature`	Crear tu rama de trabajo
2	Editar `prisma/schema.prisma`	Agregar tus cambios al schema
3	`pnpm exec prisma migrate dev --name descripcion_cambio`	Crear la migración (genera archivos en `prisma/migrations/`)
4	`pnpm exec prisma generate`	Regenerar el cliente de Prisma
5	`pnpm start:dev`	Verificar que todo funciona
6	`git add .`	Agregar TODOS los archivos (schema + migrations + src)
7	`git commit -m "feat: descripcion del cambio"`	Hacer commit con mensaje descriptivo
8	`git push origin feature/mi-feature`	Subir la rama al remoto

> ⚠️ **IMPORTANTE:** Siempre committear la carpeta `prisma/migrations/` junto con los cambios de código. Sin las migraciones, tus compañeros no pueden actualizar su base de datos.

Ejemplo de migración

Supongamos que agregas el campo `state` a `Person`:

```
// schema.prisma - antes
model Person {
  id   Int    @id @default(autoincrement())
  name String
}

// schema.prisma - después (tu cambio)
model Person {
  id    Int     @id @default(autoincrement())
  name  String
  state Boolean @default(true)  // ← nuevo campo
}
```

```
$ pnpm exec prisma migrate dev --name add_state_to_person

Prisma schema loaded from prisma/schema.prisma
Applying migration `20260310_add_state_to_person`

✔ Generated Prisma Client
```

Esto genera automáticamente el archivo:

```
prisma/migrations/20260310_add_state_to_person/migration.sql
```

---

3. Quién recibe los cambios (el compañero que hace pull)

Cuando un compañero subió cambios al schema y migraciones:

Paso	Comando	Descripción
1	`git checkout develop`	Ir a la rama base (develop o main)
2	`git pull origin develop`	Bajar los últimos cambios del remoto
3	`git checkout feature/mi-rama`	Volver a tu rama de trabajo
4	`git merge develop`	Traer los cambios de develop a tu rama
5	`pnpm exec prisma migrate deploy`	Aplicar las migraciones nuevas a tu DB local
6	`pnpm exec prisma generate`	Regenerar el cliente con los nuevos tipos
7	`pnpm start:dev`	Verificar que todo compila y funciona

> ✅ `prisma migrate deploy` aplica SOLO las migraciones pendientes que aún no están en tu base de datos. Es seguro correrlo múltiples veces.

¿Qué pasa si hay errores de TypeScript después del pull?

Si aparecen errores como `Property X does not exist on type`, el cliente de Prisma no está actualizado:

```
pnpm exec prisma generate
```

Si persiste, fuerza la regeneración:

```
rm -rf generated/prisma
pnpm exec prisma generate
```

---

4. Merge de tu rama a main o develop

Opción A: Pull Request (recomendado en equipo)

```
# 1. Asegurarte de estar actualizado
git checkout develop
git pull origin develop

# 2. Volver a tu rama y hacer merge de develop
git checkout feature/mi-feature
git merge develop

# 3. Resolver conflictos si hay (ver sección 6)

# 4. Subir tu rama actualizada
git push origin feature/mi-feature

# 5. Crear Pull Request en GitHub/GitLab
# → Tu compañero revisa y aprueba
# → Se hace merge en la plataforma
```

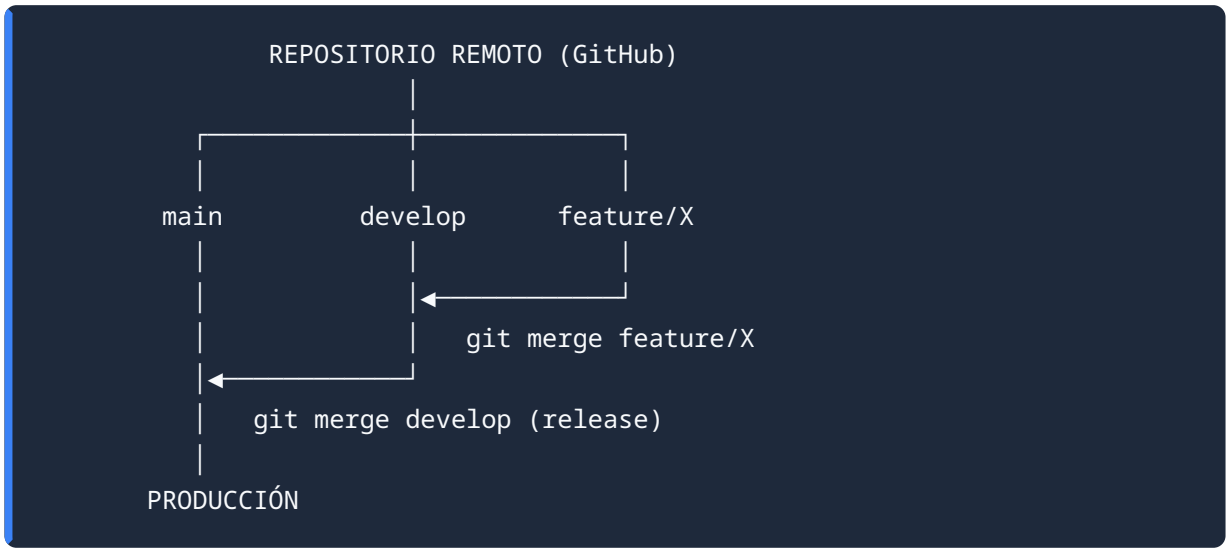
Opción B: Merge directo (proyectos personales o urgencias)

```
# 1. Ir a develop
git checkout develop

# 2. Hacer merge de tu rama
git merge feature/mi-feature

# 3. Subir develop actualizado
git push origin develop
```

> ⚠️ Después de cualquier merge que incluya cambios de schema, los compañeros DEBEN correr: `bash pnpm exec prisma migrate deploy && pnpm exec prisma generate`

---

5. Flujo completo visual



Flujo de un desarrollador:

```
1. git pull origin develop                    # bajar cambios
2. git checkout -b feature/nueva              # crear rama
3. # hacer cambios en código y schema
4. pnpm exec prisma migrate dev --name X      # crear migración
5. git add .
6. git commit -m "feat: descripción"
7. git push origin feature/nueva             # subir rama
8. # Pull Request → revisión → merge
```

Otro desarrollador (al día siguiente):

```
1. git pull origin develop                    # bajar merge
2. pnpm exec prisma migrate deploy            # aplicar migración
3. pnpm exec prisma generate                  # regenerar cliente
4. pnpm start:dev                             # verificar
```

---

6. Manejo de conflictos en schema.prisma

Si dos personas modifican el `schema.prisma` al mismo tiempo puede haber conflictos:

```
<<<<<<< HEAD (tu rama)
model Person {
  id    Int    @id
  email String  // tu campo nuevo
}
=======
model Person {
  id    Int    @id
  phone String  // campo de tu compañero
}
>>>>>>> develop
```

Solución: combinar ambos cambios manualmente:

```
model Person {
  id    Int    @id
  email String  // tu campo
  phone String  // campo del compañero
}
```

Después de resolver el conflicto:

```
pnpm exec prisma migrate dev --name merge_person_fields
git add .
git commit -m "fix: resolve schema conflict, merge email and phone"
```

---

7. Referencia rápida de comandos

Comando	Cuándo usarlo	Quién lo usa
`pnpm exec prisma migrate dev --name X`	Creé un cambio en `schema.prisma`	El que modifica
`pnpm exec prisma migrate deploy`	Recibí migraciones de otro	El que hace pull
`pnpm exec prisma generate`	Después de cualquier migrate	Todos
`pnpm exec prisma db push`	Prototipado rápido sin historial	Desarrollo individual
`pnpm exec prisma migrate status`	Ver estado de migraciones	Todos
`pnpm exec prisma studio`	Ver datos de la DB visualmente	Todos
`git pull origin develop`	Bajar cambios del remoto	Todos los días
`git checkout -b feature/X`	Empezar nueva funcionalidad	El que empieza
`git merge develop`	Traer cambios a tu rama	Antes de PR
`git push origin feature/X`	Subir tu rama al remoto	Al terminar

---

8. Reglas de oro del equipo

> Seguir estas reglas evita el 95% de los problemas en proyectos con Prisma y Git en equipo.

- **NUNCA** committear sin incluir la carpeta `prisma/migrations/`
- **SIEMPRE** correr `pnpm exec prisma migrate deploy` después de un `git pull`
- **SIEMPRE** correr `pnpm exec prisma generate` después de un migrate
- **NUNCA** hacer cambios directos en la DB sin crear una migración
- **NUNCA** borrar archivos de `prisma/migrations/` manualmente
- Un solo desarrollador crea la migración por cada feature de schema
- Nombrar las migraciones descriptivamente: `add_state_to_person`, no `migration1`
- Hacer commit de `schema.prisma` + `migration.sql` juntos, nunca por separado